CAD for VLSI

Floorplanning 2

Floorplan Representations

- Slicing
  - Normalized Polish Expression: Wong & Liu [DAC-86]
- Mosaic (and General)
  - Corner Block List (CBL): Hong et al. [ICCAD-00]
  - Q-Sequence: Sakanushi & Kajitani [APCCAS-00]
  - Twin Binary Sequence (TBS): Young, Chu, Shen [ISPD-02]
- General
  - Polar graphs: Ohtsuki et al. [ICCST-70]
  - Sequence pair: Murata et al. [ICCAD-95]
  - Bounded Slicing Grid (BSG): Nakatake [ICCAD-96]
  - Transitive Closure Graph (TCG): Lin & Chang [DAC-01]
- Compacted
  - O-tree: Guo et al. [DAC-99]
  - B*-tree: Chang et al. [DAC-00]

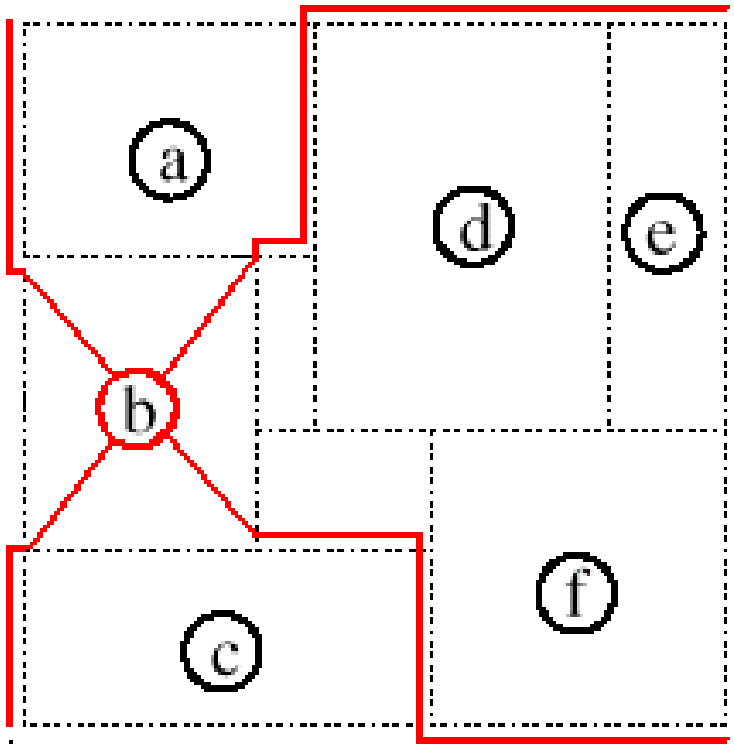
General Floorplanning by Simulated Annealing: Sequence-Pair Representation

Murata, Fujiyoshi, Nakatake, Kajitani,
“VLSI Module Placement based on Rectangle-Packing by the Sequence-Pair”,
TCAD, 15:1518-1524, December 1996.

Sequence-Pair vs. Polish Expression

- Sequence-Pair is a succinct representation of non-slicing floorplans of rectangles
  - Just like Polish Expression for slicing floorplans
- Represent a non-slicing floorplan by a pair of sequences of blocks.
- Using Simulated Annealing to find a good sequence-pair
- Can only handle hard blocks
  - i.e., cannot do things like shape-curve computation
- Essentially macro placement
- Techniques for soft block shaping exist (ex: Lagrangian Relaxation) but are very slow

Positive and Negative Step Line



Positive Step Line of Block b

Negative Step Line of Block b

Positive and Negative Step Lines

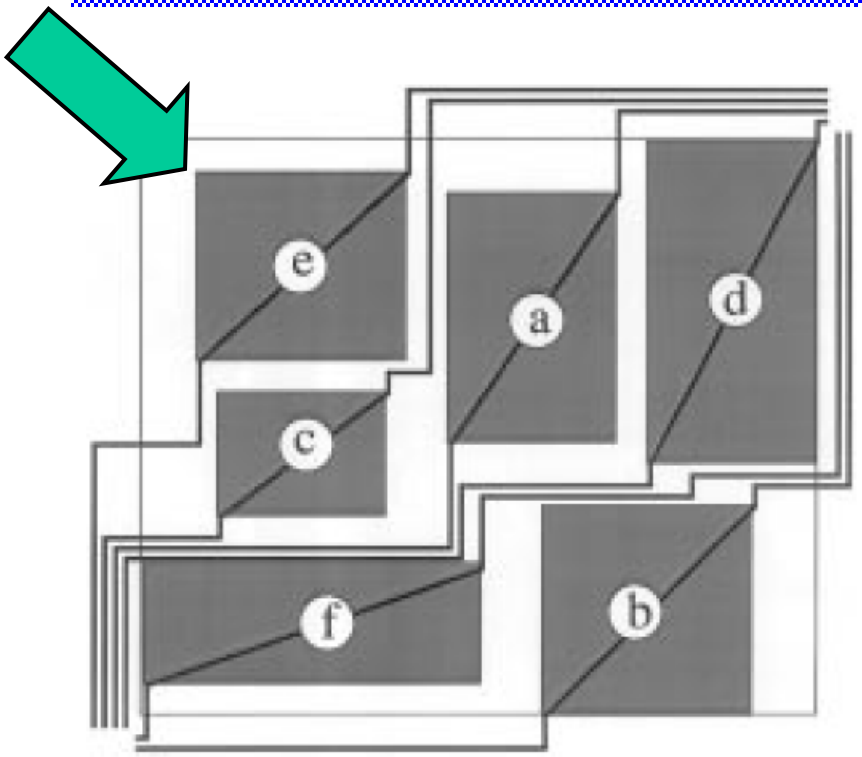
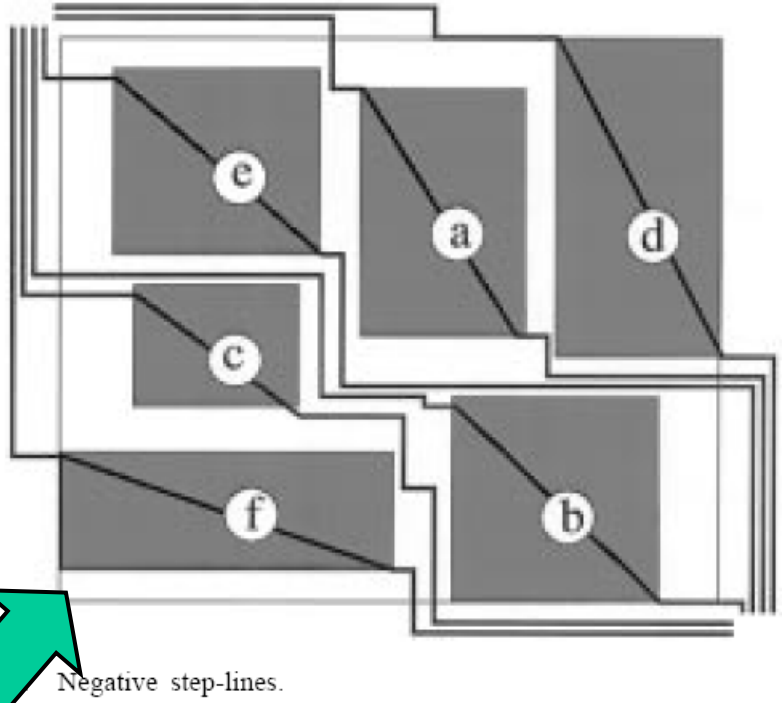


Fig. 2. Positive step-lines.

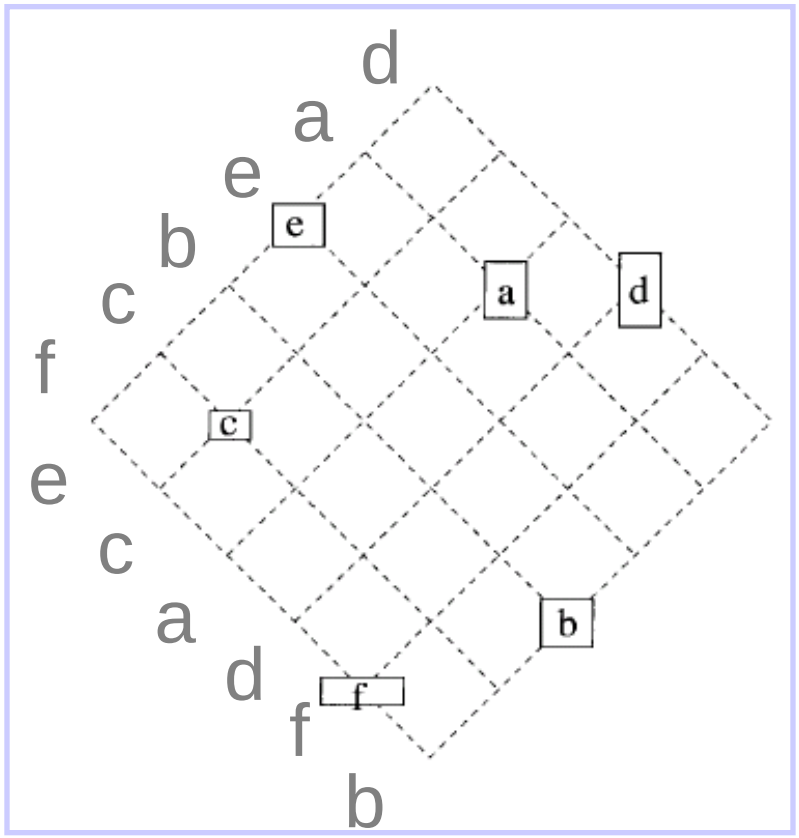


Negative step-lines.

- Positive step line sequence: ecadfb
- Negative step line sequence: fcbead

From Sequence-Pair to Floorplan

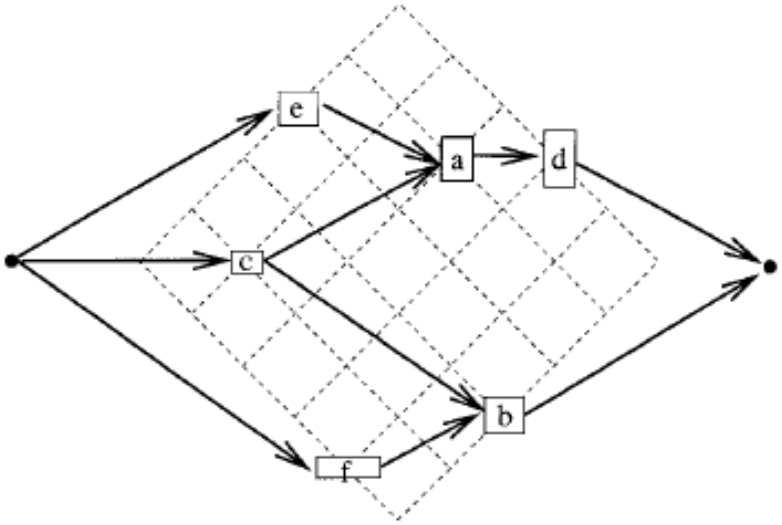
Labeled grid for
(ecadfb, bcbead)



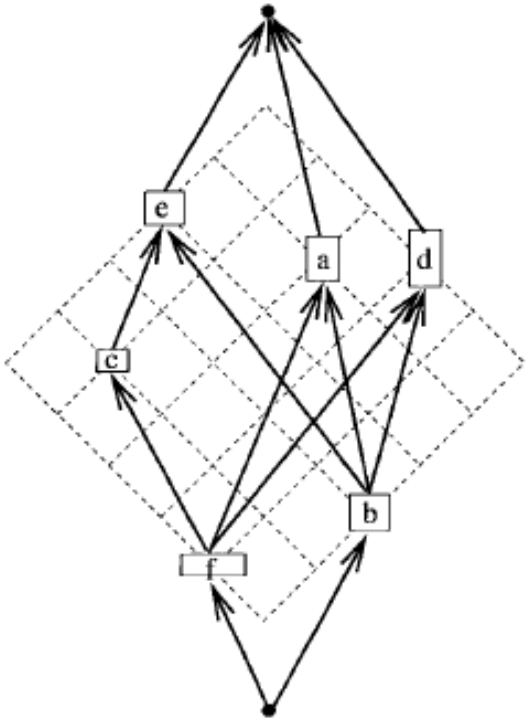
- Given a sequence-pair, the placement with smallest area can be found in $O(n^2)$ time
- Algorithms of time $O(n \log \log n)$ or $O(n \log n)$ exist. But faster than $O(n^2)$ algorithm only when n is quite large

From Sequence-Pair to Floorplan

- Distance from left (bottom) edge can be found using the longest path algorithm on the horizontal (vertical) constraint graph



Horizontal Constraint Graph



Vertical Constraint Graph

Sequence Pair (SP)

- A floorplan is represented by a pair of permutations of the module names:
  - Ex: (1 3 2 4 5, 3 5 4 1 2)

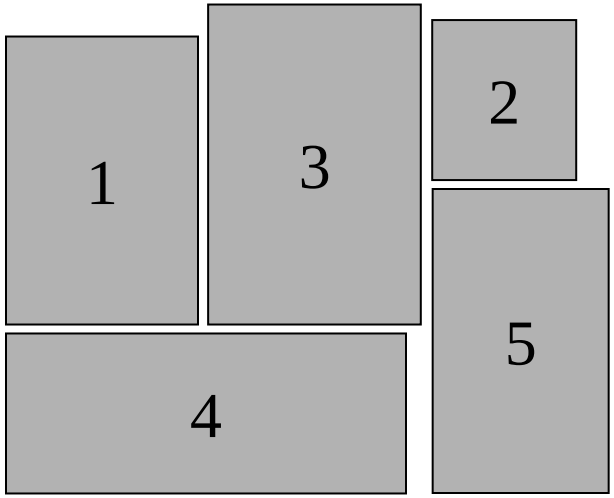
- A sequence pair (s_1, s_2) of n modules can represent all possible floorplans formed by the n modules by specifying the pair-wise relationship between the modules

Geometric Info of Sequence Pair

- Given a placement and the corresponding sequence-pair (P, N):
  - a is right to b ↔ (…b…a…, …b…a…)
  - a is left to b ↔ (…a…b…, …a…b…)
  - a is above b ↔ (…a…b…, …b…a…)
  - a is below b ↔ (…b…a…, …a…b…)

Example

- Consider the sequence pair:
  - (13245,41352)



- Any other SP that is also valid for this packing?

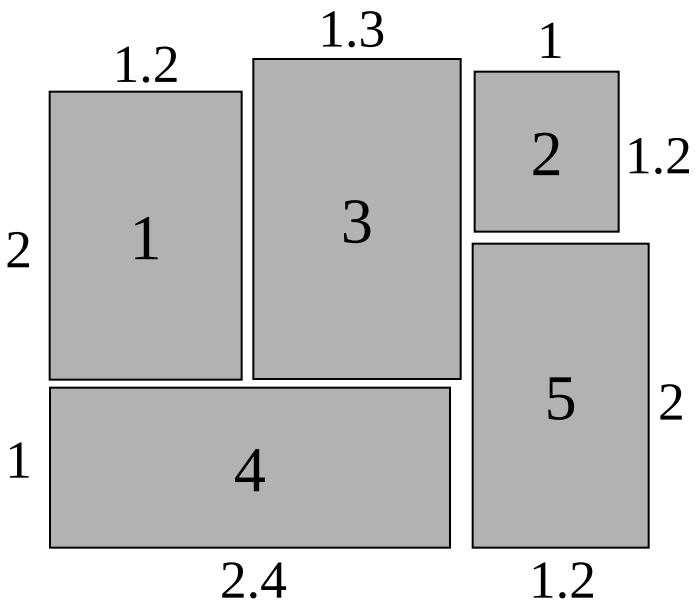
Floorplan Realization

- Floorplan realization is the step to construct a floorplan from its representation
- How to construct a floorplan from a sequence pair?
- We can make use of the horizontal and vertical constraint graphs (G_h and G_v)

Floorplan Realization

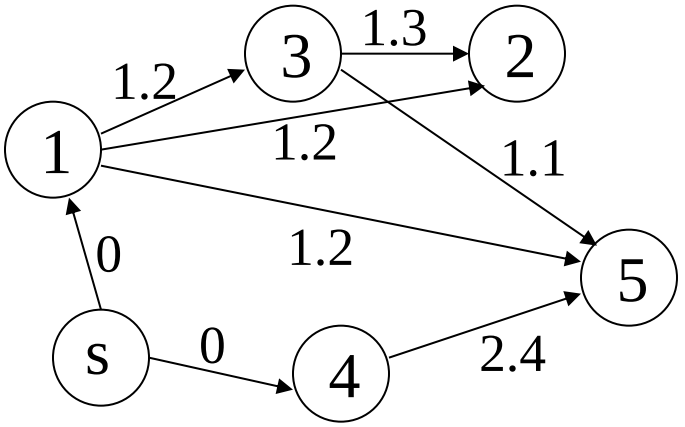
- Whenever we see (…A…B…, …A…B…), add an edge from A to B in G_h with weight w_A
- Whenever we see (…A…B…, …B…A…), add an edge from B to A in G_v with weight h_B
- Add a source vertex s to G_h and G_v pointing, with weight 0, to all vertices without incoming edges
- Finally, find the longest paths from s to every vertex in G_h and G_v (how?), which are the coordinates of the lower left corner of the module in the packing

Example

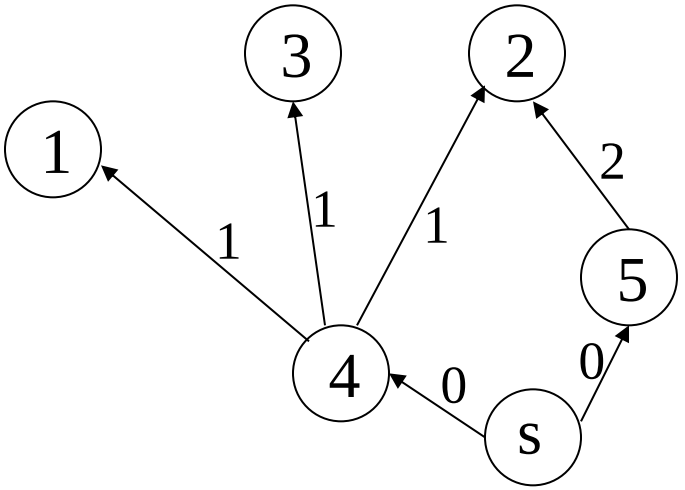


(13245,41352)

G_h



G_v



Constraint Graph

- How many edges are there in G_h and G_v in total?
- Is there any transitive edges in G_h and G_v?
- How to remove the transitive edges?
- Can we reduce the size of G_h and G_v to linear, i.e., no. of edges is of order $O(n)$, by removing all the transitive edges?

Moves

- Three kinds of moves in the annealing process:
  - M1: Rotate a module, or change the shape of a module
  - M2: Interchange 2 modules in both sequences
  - M3: Interchange 2 modules in the first sequence

- Does this set of move operations ensure reachability? Why?

Pros and Cons of SP

- Advantages:
  - Simple representation
  - All floorplans can be represented
  - The solution space is finite (How big?)
- Disadvantages:
  - Redundant representation. The representation is not 1-to-1
  - The size of the constraint graphs, and thus the runtime to construct the floorplan is quadratic

Questions

- Can we improve the runtime to realize a floorplan from its SP representation?
  - "FAST-SP: A Fast Algorithm for Block Placement on Sequence Pair", X. Tang and D.F. Wong, ASP-DAC 2001, pp. 521-526

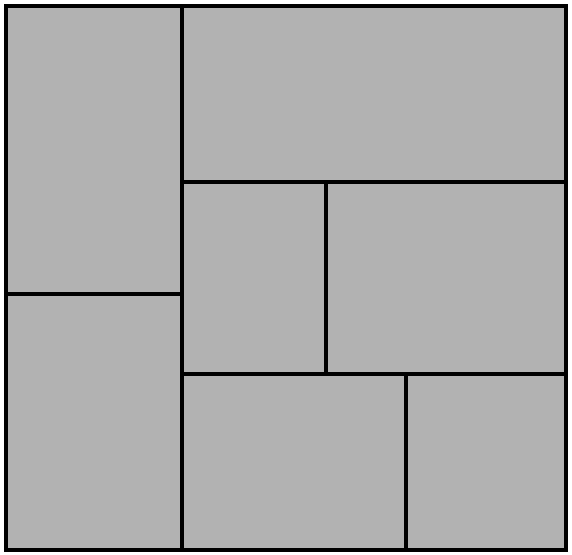
Tree-based Compacted Floorplan Representation

Chang, Chang, Wu, Wu,

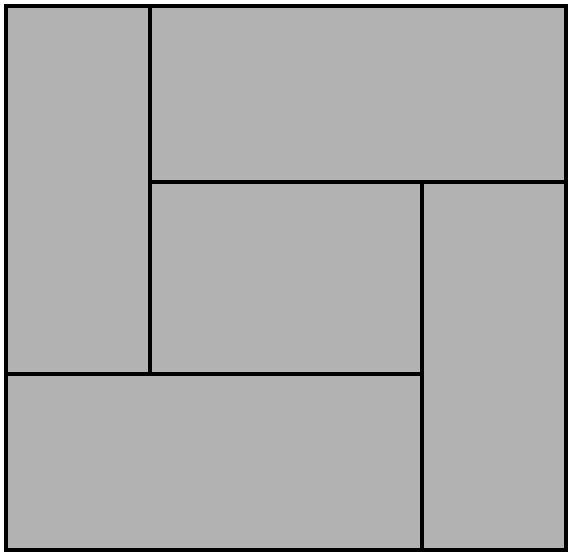
"B*-trees: A New Representation for Non-slicing Floorplans",

DAC, 458-463, June 2000.

Tree Representation

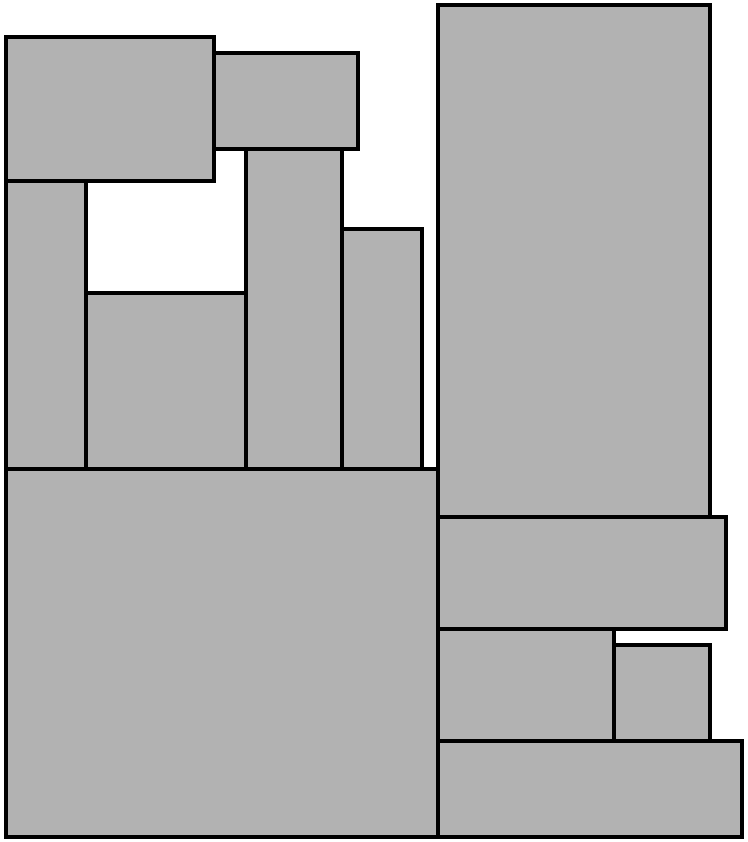


Slicing Floorplan



Non-slicing Floorplan

Tree Representation

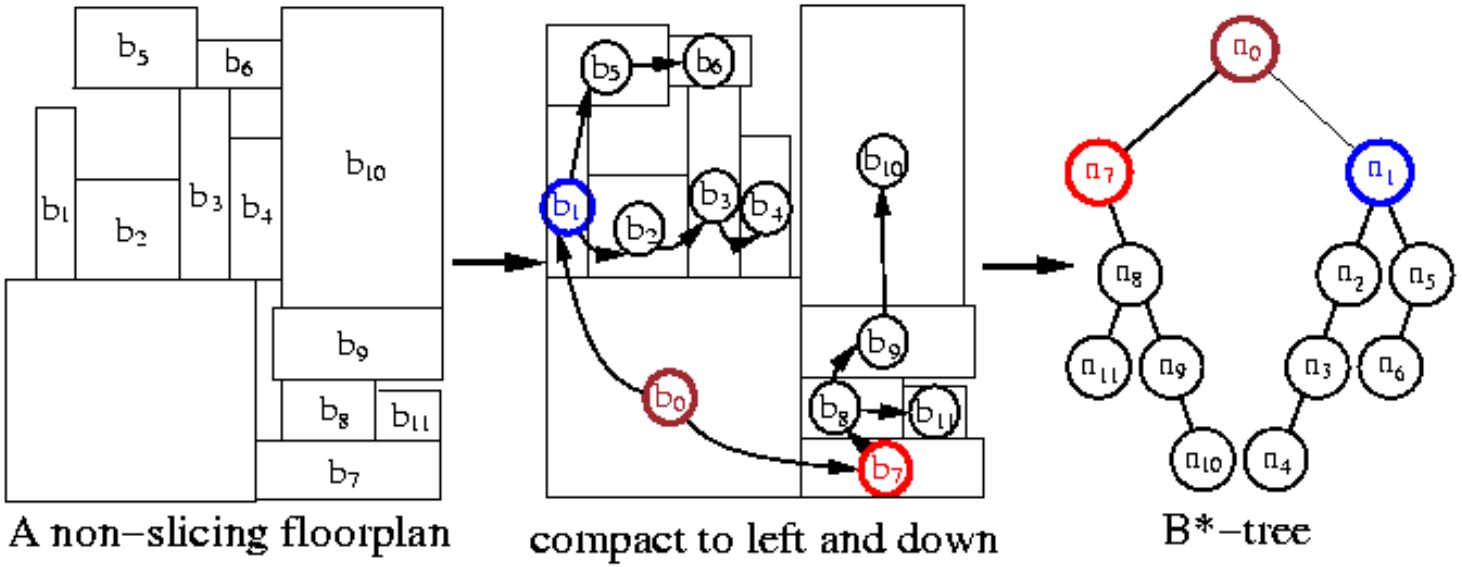


Non-slicing Floorplan

B*-Tree: Compacted Floorplan Representation

- Compact modules to left and bottom
- Construct an ordered binary tree (B*-tree)
  - Left child: the lowest, adjacent block on the right ($x_j = x_i + w_i$)
  - Right child: the first block above, with the same x-coordinate ($x_j = x_i$)

B*-tree Example



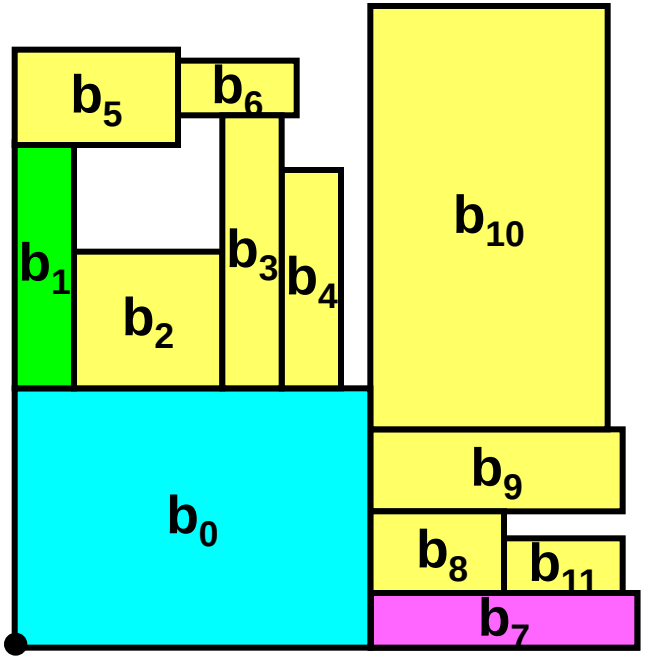
A non–slicing floorplan

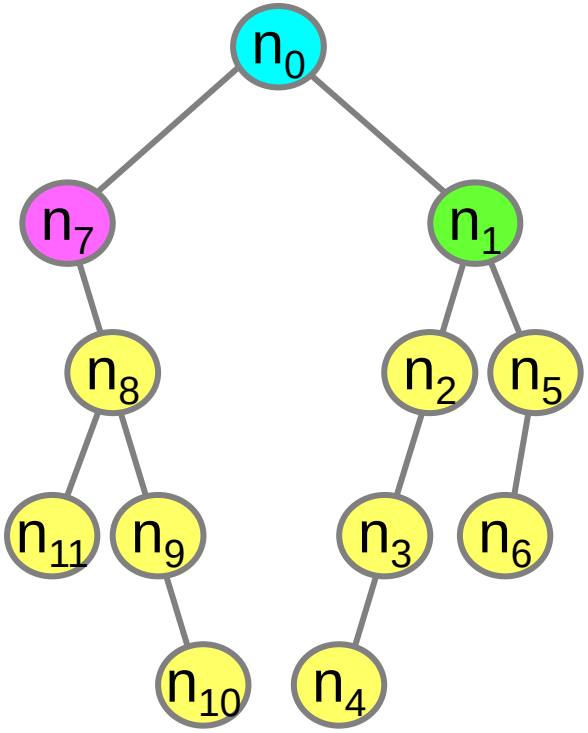
compact to left and down

B*–tree

Coordinates Computation

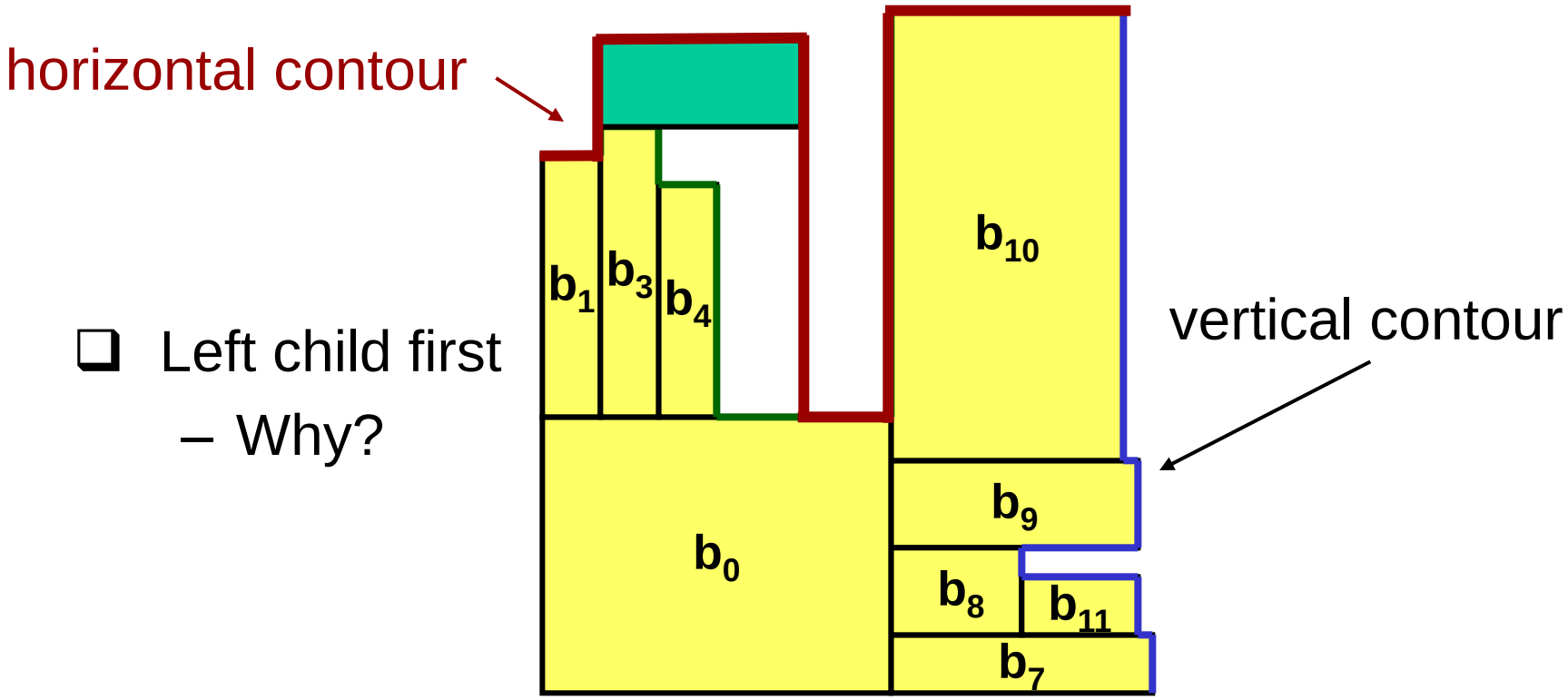
- X-coordinate of a block
- Y-coordinate of a block





Y-coordinate Computation

- Reduce the complexity of computing a y-coordinate to amortized O(1) time.



- Left child first
  - Why?

Pros and Cons of B*-tree

- Advantages
  - Binary tree based, efficient and easy
  - Transformation between a tree and its placement takes only linear time (v.s. $O(n^2)$ or $O(n\lg\lg n)$ for sequence pair)
  - Operate only on one B*-tree (v.s. 2 O-trees)
  - Smaller solution space: only $O(n!\ 4^n/n^{1.5})$ combinations (v.s. $O((n!)^2)$ for sequence pair)
  - Directly corresponds to multilevel framework for large-scale floorplan designs
- Disadvantages
  - Representation may change after packing
  - Less flexible than sequence pair in representation
    - Compacted placement only

Multilevel Floorplanning by using B*-trees

Lee, Hsu, Chang, Yang,

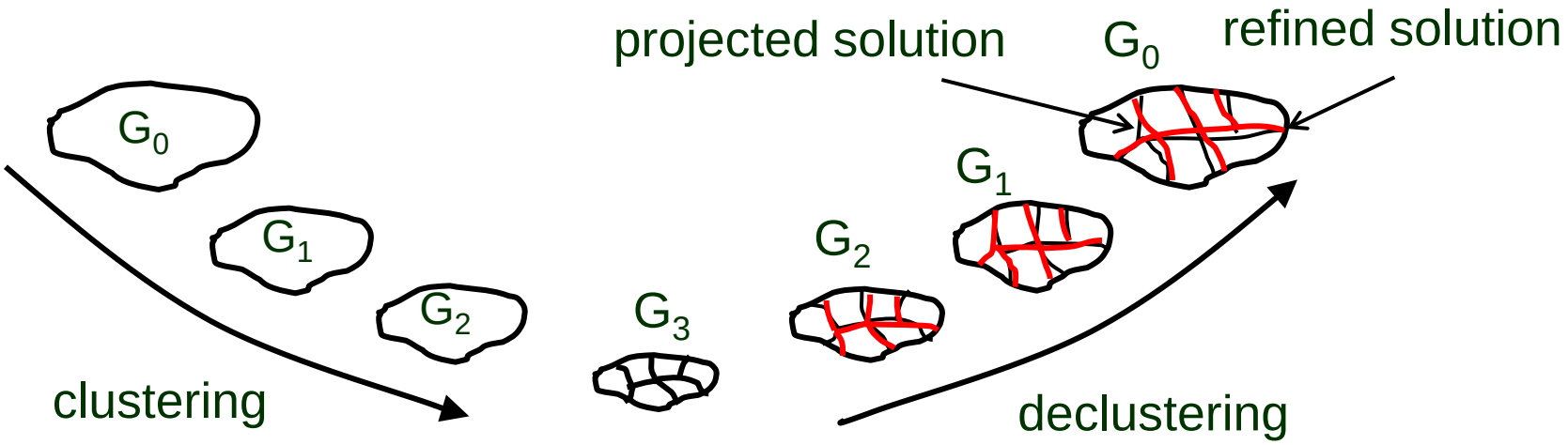
"Multilevel floorplanning/placement for large-scale modules using B*-trees",

DAC, 812-817, June 2003.

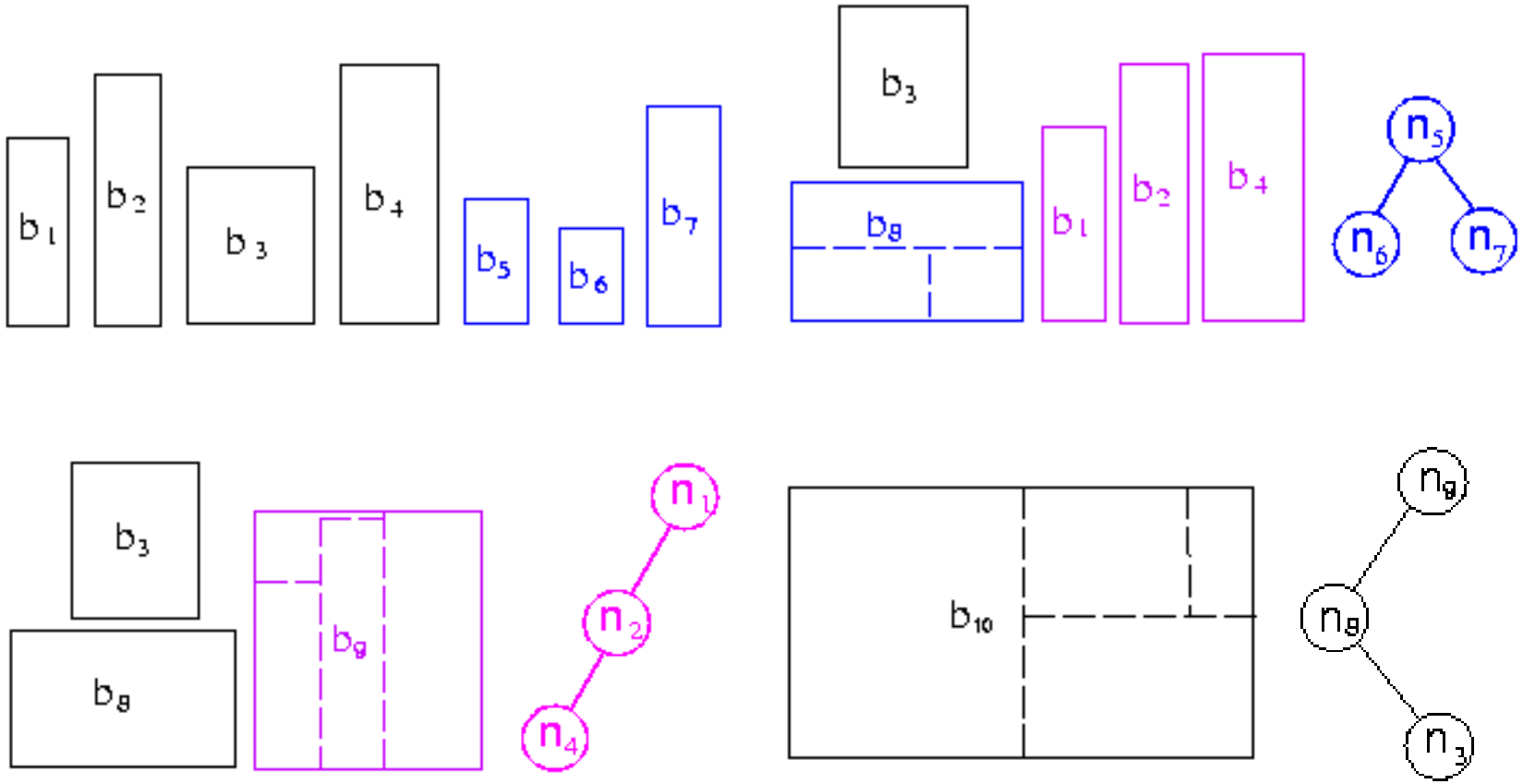
Multilevel B*-trees

- Two stages for MB*-tree: clustering followed by declustering
- Clustering
  - Iteratively groups a set of modules based on area utilization and module connectivity
  - Constructs a B*-tree to keep the geometric relations for the newly clustered modules
- Declustering
  - Iteratively ungroups a set of the previously clustered modules (i.e., perform tree expansion)
  - Refines the solution using simulated annealing

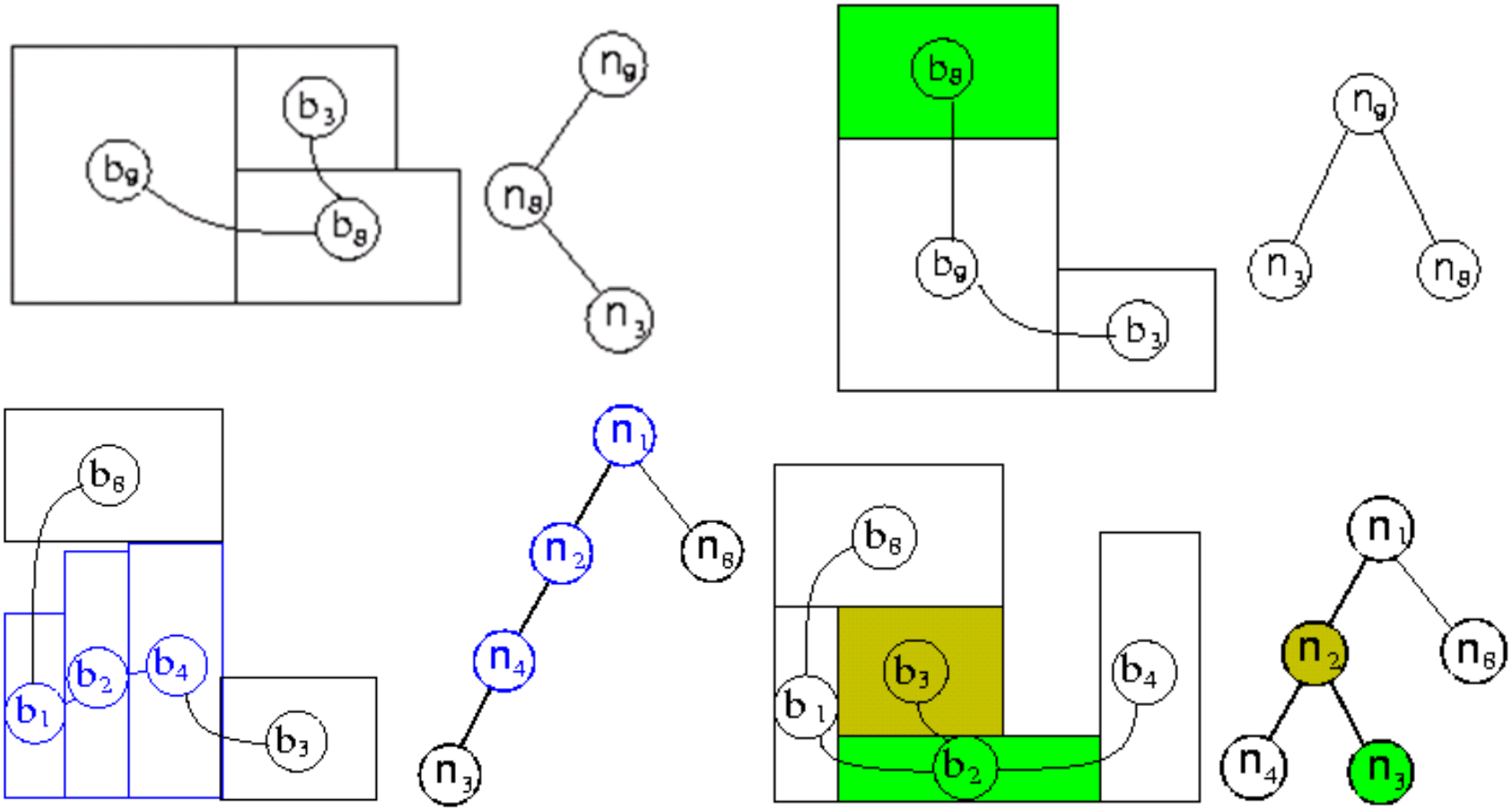
Multilevel Paradigm



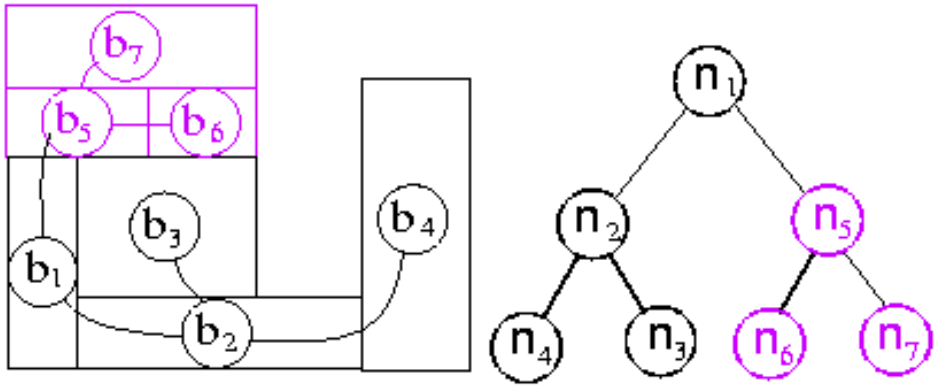
Multilevel B*-tree Example

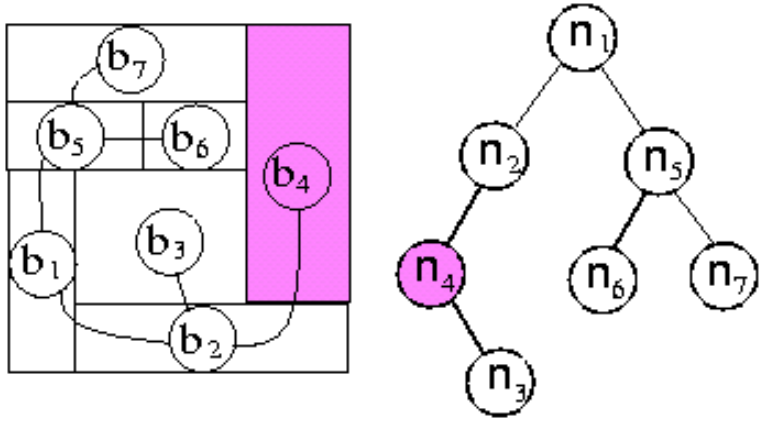


Multilevel B*-tree Example



Multilevel B*-tree Example





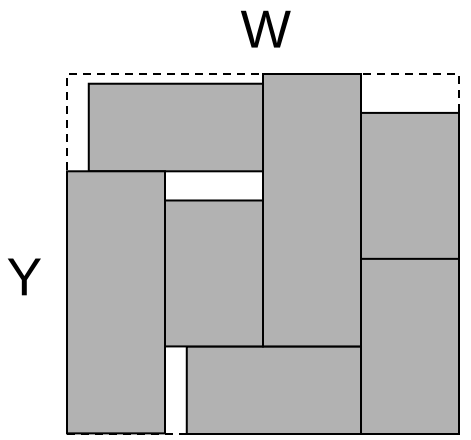
Linear Programming Approach

Sutanthavibul, Shragowitz and Rosen, “An Analytical Approach to Floorplan Design and Optimization”, IEEE Transaction on CAD, 10:761-769, June 1991.

Problem Formulation

- Minimize the packing area:
  - Assume that one dimension W is fixed
  - Minimize the other dimension Y
- Need to have constraints

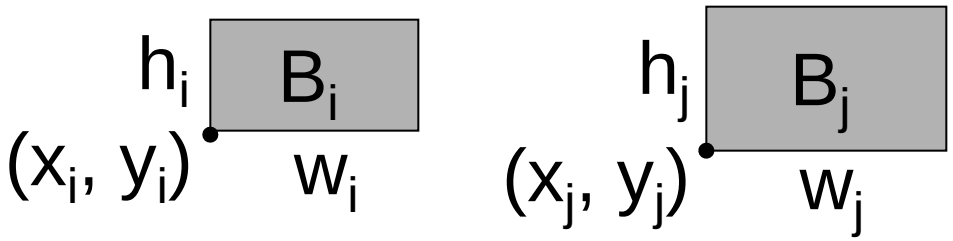
  so that blocks do not overlap
- Associate each block B_i with 4 variables:
  - x_i and y_i: coordinates of its lower left corner
  - w_i and h_i: width and height



Non-overlapping Constraints

- For two non-overlapping blocks B_i and B_j, at least one of the following four linear constraints must be satisfied

(1)	$x_i + w_i \le x_j$	if B_i is to the left of B_j
or (2)	$x_i - w_j \ge x_j$	if B_i is to the right of B_j
or (3)	$y_i + h_i \le y_j$	if B_i is below B_j
or (4)	$y_i - h_j \ge y_j$	if B_i is above B_j



Integer Variables

- Use integer (0 or 1) variables x_{ij} and y_{ij}:

  x_{ij}=0 and y_{ij} =0, if (1) is true

  x_{ij}=0 and y_{ij} =1, if (2) is true

  x_{ij}=1 and y_{ij} =0, if (3) is true

  x_{ij}=1 and y_{ij} =1, if (4) is true

- Let W and H be upper bounds on the total width and height. Non-overlapping constraints:

$$
\begin{aligned}
&(1') \quad x_i + w_i \le x_j + W(x_{ij} + y_{ij}) \\
&(2') \quad x_i - w_j \ge x_j - W(1 + x_{ij} - y_{ij}) \\
&(3') \quad y_i + h_i \le y_j + H(1 - x_{ij} + y_{ij}) \\
&(4') \quad y_i - h_j \ge y_j - H(2 - x_{ij} - y_{ij})
\end{aligned}
$$

Formulation

$$
\begin{array}{lll}
\text{Min.} & Y & \\
\text{s.t.} & 0 \le x_i,\ x_i + w_i \le W & 1 \le i \le n \\
 & 0 \le y_i,\ y_i + h_i \le Y & 1 \le i \le n \\
 & x_i + w_i \le x_j + W(x_{ij} + y_{ij}) & 1 \le i < j \le n \\
 & x_i - w_j \ge x_j - W(1 + x_{ij} - y_{ij}) & 1 \le i < j \le n \\
 & y_i + h_i \le y_j + H(1 - x_{ij} + y_{ij}) & 1 \le i < j \le n \\
 & y_i - h_j \ge y_j - H(2 - x_{ij} - y_{ij}) & 1 \le i < j \le n \\
 & x_{ij} = 0 \text{ or } 1 & 1 \le i < j \le n \\
 & y_{ij} = 0 \text{ or } 1 & 1 \le i < j \le n
\end{array}
$$

Formulation with Rotation

- If the blocks can be rotated, use a 0-1 integer variable z_i for each block B_i s.t. $z_i = 0$ if B_i is in the original orientation and $z_i = 1$ if B_i is rotated 90°

$$
\begin{array}{lll}
\text{Min.} & Y & \\
\text{s.t.} & 0 \le x_i,\ x_i + z_i h_i + (1 - z_i) w_i \le W & 1 \le i \le n \\
 & 0 \le y_i,\ y_i + z_i w_i + (1 - z_i) h_i \le Y & 1 \le i \le n \\
 & x_i + z_i h_i + (1 - z_i) w_i \le x_j + W(x_{ij} + y_{ij}) & 1 \le i < j \le n \\
 & x_i - z_j h_j - (1 - z_j) w_j \ge x_j - W(1 + x_{ij} - y_{ij}) & 1 \le i < j \le n \\
 & y_i + z_i w_i + (1 - z_i) h_i \le y_j + H(1 - x_{ij} + y_{ij}) & 1 \le i < j \le n \\
 & y_i - z_j w_j - (1 - z_j) h_j \ge y_j - H(2 - x_{ij} - y_{ij}) & 1 \le i < j \le n \\
 & x_{ij} = 0 \text{ or } 1 & 1 \le i < j \le n \\
 & y_{ij} = 0 \text{ or } 1 & 1 \le i < j \le n \\
 & z_{ij} = 0 \text{ or } 1 & 1 \le i < j \le n
\end{array}
$$

Formulation with Soft Blocks

- If B_i is a soft block, $w_i h_i = A_i$. But this constraint is quadratic!

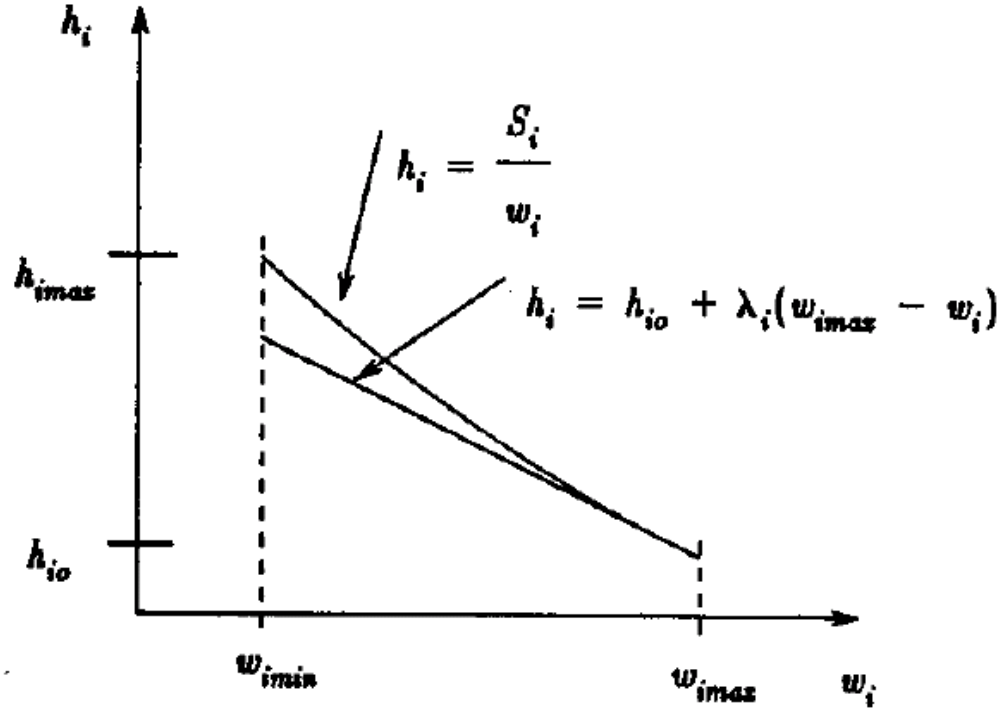


Fig. 1. Approximation function for h_i.

Formulation with Soft Blocks

- Linearized by taking the first two terms of the Taylor expression of $h_i=A_i/w_i$ at w_{imax} (max. width of block B_i)

$$h_i = h_{imin}+\lambda_i(w_{imax}-w_i)$$

where $h_{imin} = A_i/w_{imax}$ and $\lambda_i = A_i/w_{imax}^2$

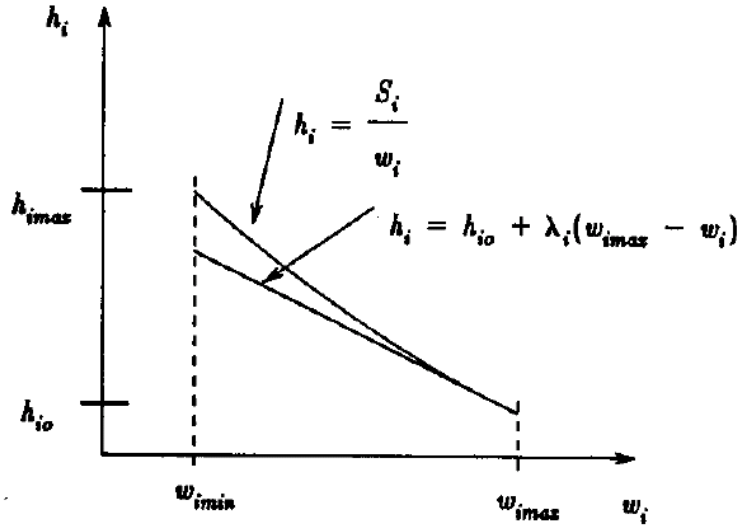


Fig. 1. Approximation function for h_i.

Formulation with Soft Blocks

- If B_i is soft and B_j is hard:

$$
\begin{aligned}
&(1) \quad x_i + w_i \le x_j + W(x_{ij} + y_{ij}) \\
&(2) \quad x_i - w_j \ge x_j - W(1 + x_{ij} - y_{ij}) \\
&(3) \quad y_i + h_{imin} + \lambda_i (w_{imax} - w_i) \le y_j + H(1 - x_{ij} + y_{ij}) \\
&(4) \quad y_i - h_j \ge y_j - H(2 - x_{ij} - y_{ij})
\end{aligned}
$$

- If both B_i and B_j are soft:

$$
\begin{aligned}
&(1) \quad x_i + w_i \le x_j + W(x_{ij} + y_{ij}) \\
&(2) \quad x_i - w_j \ge x_j - W(1 + x_{ij} - y_{ij}) \\
&(3) \quad y_i + h_{imin} + \lambda_i (w_{imax} - w_i) \le y_j + H(1 - x_{ij} + y_{ij}) \\
&(4) \quad y_i - h_{jmin} - \lambda_j (w_{jmax} - w_j) \ge y_j - H(2 - x_{ij} - y_{ij})
\end{aligned}
$$

Another Way to Linearize

- Assumptions: w_i, h_i are unknown; area lower bound: A_i.
- Module size constraints: $w_i h_i \geq A_i$; $a_i \leq \frac{w_i}{h_i} \leq b_i$.
- Hence, $w_{min} = \sqrt{A_i a_i}$, $w_{max} = \sqrt{A_i b_i}$, $h_{min} = \sqrt{\frac{A_i}{b_i}}$, $h_{max} = \sqrt{\frac{A_i}{a_i}}$.
- $w_i h_i \geq A_i$ nonlinear! How to fix?
  - Can apply a first-order approximation of the equation: a line passing through (w_{min}, h_{max}) and (w_{max}, h_{min}).

$$h_i = \Delta_i w_i + c_i \qquad /* \ y = mx + c \ */$$
$$\Delta_i = \frac{h_{max} - h_{min}}{w_{min} - w_{max}} \qquad /* \ slope \ */$$
$$c_i = h_{max} - \Delta_i w_{min} \qquad /* \ c = y_0 - mx_0 \ */$$

  - Substitute $\Delta_i w_i + c_i$ for h_i to form linear constraints (x_i, y_i, w_i are unknown; Δ_i, c_i, can be computed as above).

Solving Linear Programming

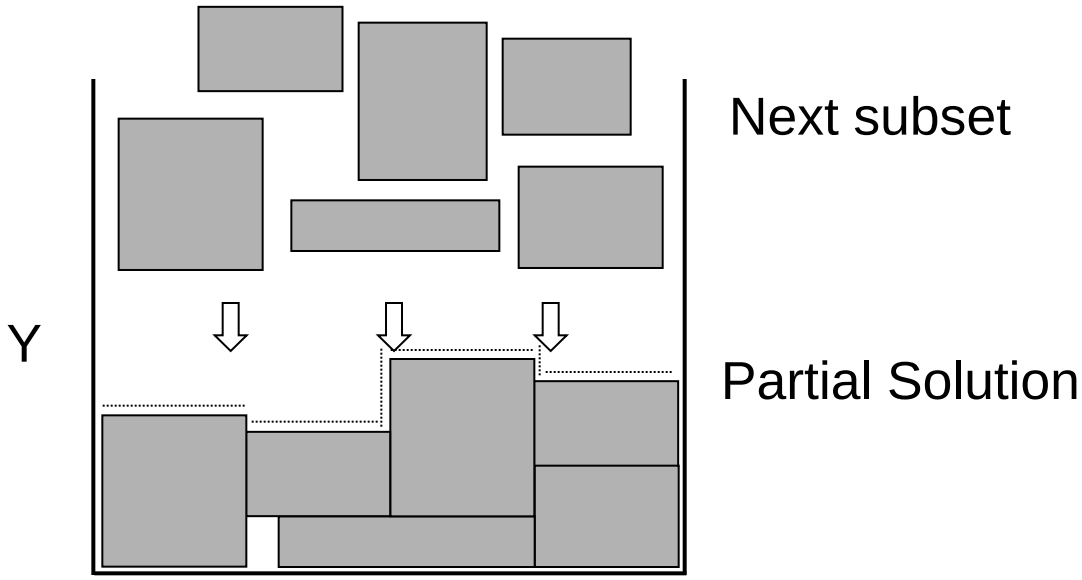
- Linear Programming (LP) can be solved by classical optimization techniques in polynomial time
- Mixed Integer LP (MILP) is NP-Complete
  - The run time of the best known algorithm is exponential to the number of variables and equations

Complexity

- For a problem with n blocks, and for the simplest case, i.e., all blocks are hard:
  - 2n continuous variables (x_i, y_i)
  - 2n(n-1) +n integer variables (x_{ij}, y_{ij}, z_i)
  - $4n^2$-2n linear constraints
- Practically, this method can only solve small size problems

Successive Augmentation

- A classical greedy approach to keep the problem size small: repeatedly pick a small subset of blocks to formulate a MILP, solve it together with the previously picked blocks with fixed locations and shapes:



Summary of Floorplanning

- Stockmeyer
  - Slicing with given set of modules
  - Dynamic programming (only keep irredundant solutions)
- Wong-Liu
  - Slicing floorplan
  - Nice polish expression
- Sequence pair
  - Nonslicing
  - Nice compact representation
- B*-tree
  - Binary tree representation
  - Multilevel framework
- Mixed Integer Linear Programming
  - Nonslicing
  - Not scalable

